Universidad Técnica Nacional

Calidad de Software

Mini laboratorio

Estudiantes:

Jacqueline Molina Montero

208490403

Daniel Nuñez Hurtado

208240809

26/03/2026

V-01 — SQL Injection en Login

Descripción del problema

La ruta /login construía la consulta SQL concatenando directamente los valores del formulario en un f-string. Esto permite inyectar sentencias SQL arbitrarias y omitir la verificación de credenciales.

```
# VULNERABLE
query = f"SELECT * FROM users WHERE username = '{username}' AND password =
'{password}'"
```

Evidencia — Ataque 1: Bypass con admin' --

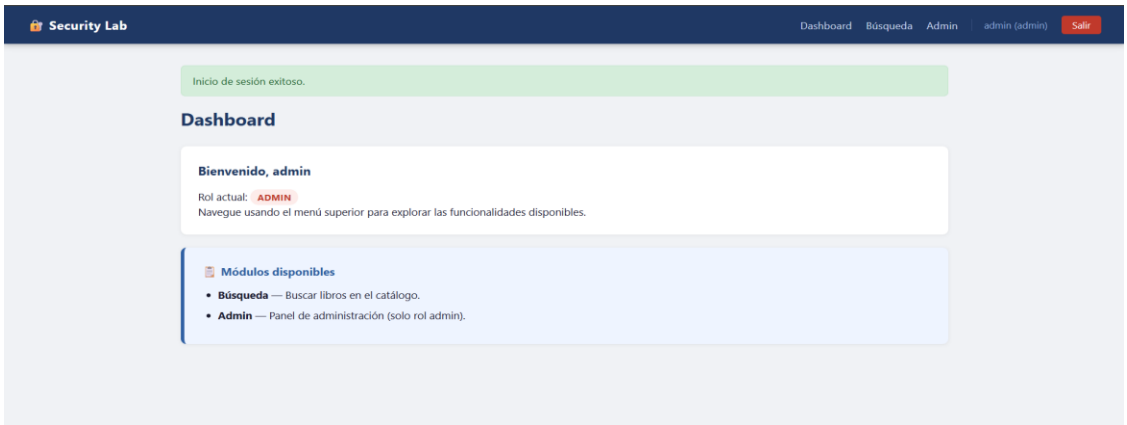
Payload en el campo Usuario: admin' --

El doble guión comenta el resto del SQL, ignorando la condición de contraseña.

Evidencia — Ataque 2: Login sin conocer usuario

Payload en el campo Usuario: ' OR '1'='1' --

La condición WHERE es siempre verdadera, devolviendo el primer usuario de la tabla.



Por qué es peligrosa

Un atacante sin credenciales puede acceder como cualquier usuario, incluyendo el administrador. También permite enumerar, modificar o eliminar datos.

Corrección aplicada

```
# CORREGIDO
user = conn.execute(
    "SELECT * FROM users WHERE username = ?",
    (username,)
).fetchone()
```

Consulta parametrizada con ?. El driver de SQLite escapa automáticamente el input del usuario.

V-02 — SQL Injection en Búsqueda (UNION)

Descripción del problema

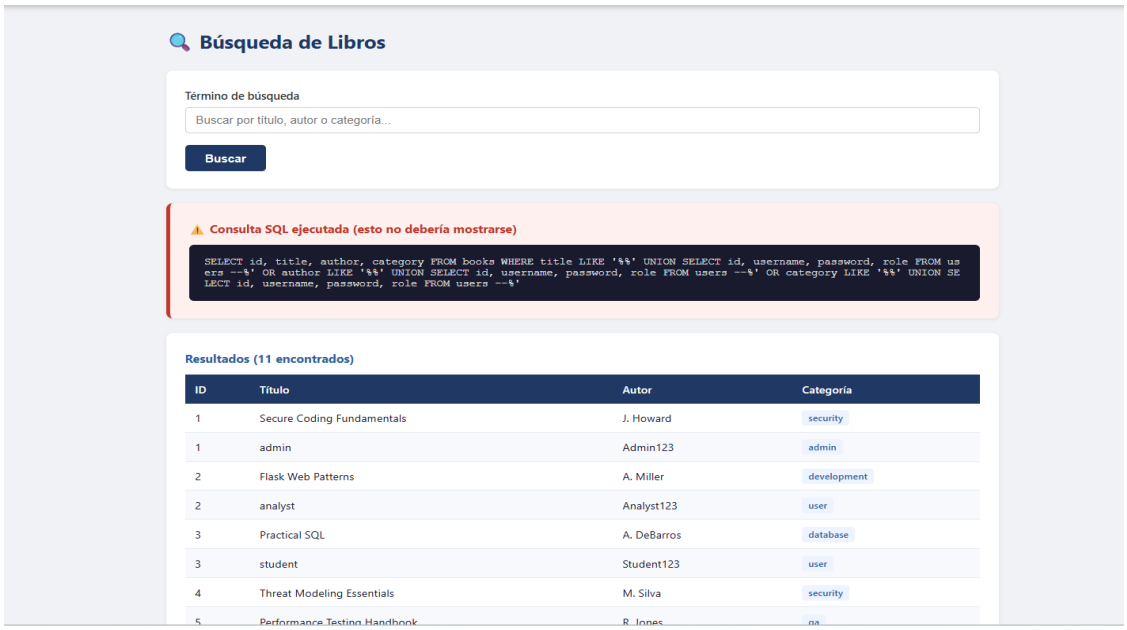
La ruta /search construía la consulta LIKE concatenando el término directamente, permitiendo inyectar una cláusula UNION para extraer datos de otras tablas.

```
# VULNERABLE
raw_query = f"SELECT id, title, author, category FROM books
            WHERE title LIKE '%{term}%' ..."
```

Evidencia — Ataque 3: Extracción de usuarios con UNION

Payload: %' UNION SELECT id, username, password, role FROM users --

Los resultados muestran los usuarios con sus contraseñas en texto plano.



Por qué es peligrosa

Permite exfiltrar el contenido completo de cualquier tabla. En este caso se extrajeron credenciales, pero la técnica se extiende a cualquier dato almacenado.

Corrección aplicada

```
# CORREGIDO
books = conn.execute(
    "SELECT id, title, author, category FROM books
     WHERE title LIKE ? OR author LIKE ? OR category LIKE ?",
    (f"%{term}%", f"%{term}%", f"%{term}%")
).fetchall()
```

El comodín % se agrega en Python. El driver escapa los caracteres especiales del término.

V-03 — Contraseñas en Texto Plano

Descripción del problema

Las contraseñas se almacenaban y comparaban como texto sin hashing. Cualquier acceso a la base de datos revelaba las contraseñas reales de todos los usuarios.

```
# VULNERABLE
("admin", "Admin123", "admin"),

if user:  # solo verifica existencia, sin comparar hash
```

Evidencia — Contraseñas visibles en el panel Admin

Por qué es peligrosa

Si un atacante obtiene acceso a la base de datos consigue todas las contraseñas de inmediato. Las contraseñas suelen reutilizarse en otros servicios, amplificando el impacto.

Corrección aplicada

```
# CORREGIDO
from werkzeug.security import generate_password_hash, check_password_hash

# Al insertar:
("admin", generate_password_hash("Admin123"), "admin"),

# Al validar:
if user and check_password_hash(user["password"], password):
```

werkzeug implementa PBKDF2-SHA256 con sal aleatoria. Cada hash es único e irreversible.

V-04 — SECRET_KEY Hardcodeada

Descripción del problema

La clave secreta de Flask estaba escrita directamente en el código fuente. Cualquier persona con acceso al repositorio podía leerla y usarla para falsificar sesiones.

```
# VULNERABLE
app.config["SECRET_KEY"] = "dev-secret-key-insegura-1234"
```

Por qué es peligrosa

La SECRET_KEY firma las cookies de sesión. Conociéndola, un atacante puede forjar cookies válidas con cualquier usuario o rol, obteniendo acceso de administrador sin credenciales.

Corrección aplicada

```
# CORREGIDO
app.config["SECRET_KEY"] = os.environ.get("FLASK_SECRET_KEY", "")
if not app.config["SECRET_KEY"]:
    raise RuntimeError("FLASK_SECRET_KEY no está definida.")

# En docker-compose.yml:
environment:
  - FLASK_SECRET_KEY=clave-aleatoria-larga-y-segura
```

La clave se carga desde variable de entorno y nunca queda registrada en el repositorio.

V-05 — Modo debug=True Activo

Descripción del problema

El servidor Flask arrancaba con debug=True hardcodeado. En modo debug, ante cualquier error, Flask expone un debugger interactivo en el navegador.

```
# VULNERABLE
app.run(host="0.0.0.0", port=5000, debug=True)
```

Por qué es peligrosa

El debugger interactivo de Werkzeug permite ejecutar expresiones Python en el servidor. Un atacante con acceso puede leer archivos del sistema, ejecutar comandos y comprometer el servidor completamente.

Corrección aplicada

```
# CORREGIDO
debug_mode = os.environ.get("FLASK_DEBUG", "0") == "1"
app.run(host="0.0.0.0", port=5000, debug=debug_mode)

# En docker-compose.yml (producción):
  - FLASK_DEBUG=0
```

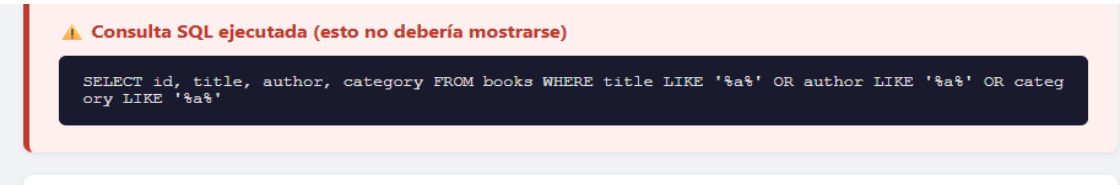
El modo debug es controlado por variable de entorno. Por defecto es 0 (apagado).

V-06 — Consulta SQL Expuesta en Pantalla

Descripción del problema

El template search.html recibía la variable raw_query con el SQL completo ejecutado y lo mostraba bajo el formulario de búsqueda.

Evidencia



Por qué es peligrosa

Exponer la estructura del SQL revela nombres de tablas y columnas. Esta información es esencial para construir ataques UNION como el de V-02.

Corrección aplicada

```
<!-- ELIMINADO de search.html -->
{% if raw_query %}
<div class="vuln-card">
  <pre>{{ raw_query }}</pre>
</div>
{% endif %}
```

Se eliminó la variable raw_query en app.py y el bloque completo en search.html.

V-07 — Enlace Admin Visible para Todos los Roles

Descripción del problema

El layout mostraba el enlace al panel Admin para todos los usuarios autenticados, sin importar su rol.

```
<!-- VULNERABLE -->
<a href="{{ url_for('admin') }}">Admin</a>
```

Por qué es peligrosa

Revelar recursos de administración a usuarios sin privilegios aumenta la superficie de ataque y facilita intentos de escalada de privilegios.

Corrección aplicada

```
<!-- CORREGIDO en layout.html -->
{% if session.get("role") == "admin" %}
  <a href="{{ url_for('admin') }}">Admin</a>
{% endif %}
```

El enlace solo se renderiza cuando el rol en sesión es admin.

V-08 — Formularios sin Protección CSRF

Descripción del problema

Los formularios de login y búsqueda no incluían token CSRF, permitiendo que páginas externas envíen solicitudes POST en nombre de usuarios autenticados.

```
<!-- VULNERABLE -->
<form method="post">
  <input name="username"> ...
</form>
```

Por qué es peligrosa

Un atacante puede alojar una página con un formulario oculto apuntando a la aplicación. Si la víctima la visita con sesión activa, el navegador envía las cookies automáticamente y la acción se ejecuta como si fuera legítima.

Corrección aplicada

```
# app.py
from flask_wtf.csrf import CSRFProtect
csrf = CSRFProtect(app)

<!-- login.html y search.html -->
<form method="post">
  <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
  ...
</form>
```

Flask-WTF rechaza cualquier POST sin token válido con error 400 Bad Request — The CSRF token is missing.

Conclusiones

- La vulnerabilidad más crítica fue V-01 (SQLi Login), ya que permite acceder como administrador sin conocer ninguna credencial, comprometiendo completamente el sistema de autenticación.
- Ningún dato proveniente del usuario debe concatenarse directamente en consultas SQL. Las consultas parametrizadas son la solución estándar y no tienen costo adicional en rendimiento.
- Las contraseñas nunca deben almacenarse en texto plano. El uso de hashing con sal (PBKDF2 vía werkzeug) es el mínimo aceptable en cualquier aplicación real.
- Los secretos como SECRET_KEY no deben aparecer en el código fuente ni en el repositorio. Las variables de entorno son el mecanismo correcto para inyectarlos en tiempo de ejecución.
- Lo más difícil fue la corrección de V-03: al cambiar el esquema de almacenamiento fue necesario eliminar y regenerar la base de datos para que los hashes fueran consistentes con el nuevo código.
- La exposición de información interna (V-06) demuestra cómo un problema que no parece crítico por sí solo puede ser el eslabón que facilita ataques mucho más graves como el UNION de V-02.